1. **[15 pts]** True or False. Give short reasons for your answers:

   (a) A classifier trained with a lower training rate is less likely to overfit.

   (b) Both linear regression and logistic regression are special cases of neural networks.

   (c) The MLE estimate for the parameters (α_1, α_2) of the following model can be computed analytically: $y_i = \log(x_i^{\alpha_1} e^{\alpha_2}) + \epsilon_i$, (where $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$ is i.i.d noise).

   (d) If you increase the number of hidden layers in a multi layer perceptron, the classification error of test data always decreases.

   (e) There's no difference between applying the activation function after applying dropout, or applying the activation function before dropout.

   (f) Cross-entropy loss can take on negative values.

   Solution:

   (a) False. Overfitting isn't related to the training rate. Training rate is determined by the size / complexity of our model and the quantity of data available.

   (b) True. Linear Regression is just an MLP with no hidden layers. Logistic regression is an MLP with no hidden layers with a sigmoid activation on the output layer.

   (c) True. The model is linear in terms of (α_1, α_2): $y_i = \log(x_i^{\alpha_1} e^{\alpha_2}) + \epsilon_i = y_i = \alpha_1 \log(x_i) + \alpha_2 + \epsilon_i$.

   (d) False. Increasing model complexity may lead to overfitting on the training data and this lead to worse performance on the test set.

   (e) True / False (justification matters): Dropout would make certain neurons to zero. If we consider activation functions that map 0 to 0, then the statement holds true. Otherwise, the output of the neuron would be $\phi(0)$, where, ϕ is the activation function considered.

   (f) False. The cross-entropy loss is given by $-\sum p_i \log(\hat{p}_i)$. We know that $\hat{p}_i \in [0, 1] \implies \log(\hat{p}) < 0$. Also, we know that $p_i \in \{0, 1\}$. So the cross-entropy loss must always be positive.

4. **[15 pts]** Provide brief answers to the following questions:

   (a) **[7.5 pts]** Briefly explain what is the "re-parameterization trick" that is utilized in the context of training variational models.

   (b) **[7.5 pts]** What do we mean by "exploding" and "vanishing" gradients? Explain how this problem is mitigated in RNNs.

Solution:

(a) You need to explain how we can reformulate our gradient estimation so that the gradient is not related to the distribution over which we take the expectation. That is,

$$\nabla_\phi \mathbb{E}_{z\sim q(z|x)}[f(z)] = \mathbb{E}_{\epsilon\sim p(\epsilon)}[\nabla_\phi f(h(\epsilon,\phi))] \tag{14}$$

Refer to pages 59-60 of the notes.

(b) When training a neural network by backpropagation, we compute gradients by the chain rule. For activation functions which have a small gradient value, this has the effect of multiplying n of these small numbers to compute gradients of the early layers in an n-layer network, meaning that the gradient decreases exponentially with n while the early layers train very slowly. When activation functions are used whose derivatives can take on larger values, one risks encountering the related exploding gradient problem. For RNNs, vanishing gradients can be reduced by using the Long Short-Term Memory (LSTM) memory units and related gated-type neuron structures. For exploding gradients, we can also utilize gradient-clipping where we check for and limit the size of gradients during the training of the network.

I dont think variational models have been discussed yet so ignore 4a

Question 1 (10pts)

Please answer the following True or False questions with some explanation.

0.1 (a) (3pts)

"After your training is done, the testing error appears less than your training error."

0.2 (b) (3pts)

"Increase the number of training examples in logistic regression will eventually decrease the Bias and increase the Variance of the predictions."

0.3 (c) (4pts)

"L_2 regularization is having better property to impose sparsity than L_1 regularization."

0.1a: False, you should generally expect your testing errors to be worse than you training error or at most, close to them.

0.2b: Adding more training examples mainly decreases variance, because with more data the fitted model depends less on any particular sample. Bias stays roughly the same, since bias comes from the model's assumptions (logistic regression is a linear decision boundary), and more data can't make a linear model capture a nonlinear pattern. So the statement has both directions wrong for variance and is wrong about bias decreasing.

0.3c: False L^2 regularization (Ridge) adds the term $\lambda||w||_2^2$ which enforces a small 2-norm of the weights, this shrinks the weights toward zero but rarely makes them exactly zero. This is because the gradient of this term wrt w is $2\lambda w$ so as w gets smaller so does its gradient and the pull towards zero is proportional to the weights itself. L^1 regularization (Lasso) add the term $\lambda||w||_1$ which can drive w to zero and produces sparsity. This is because the gradient of this term wrt w is $\lambda \text{sign}(w) = \pm\lambda$, so regardless of the magnitude of w the strength of the pull towards zero stays constant.

1. **[15 pts]** True or False. Give short reasons for your answers:

   (a) A classifier trained with a second-order optimizer such as Newton's method is less likely to overfit.

   (b) The gradient descent algorithm is guaranteed to find a local minimum if the learning rate is correctly decreased and a finite local minimum exists..

   (c) A single perceptron (no hidden layer) can be used to learn an exclusive-OR (XOR) gate. (an XOR gate can be represented as $f(a, b) = |a - b|$)

   (d) If you increase the number of neurons in the hidden layers of a multi layer perceptron, the classification error of test data always decreases.

   (e) KL Divergence is a symmetric measure of similarity (that is $\mathbb{KL}(P||Q) == \mathbb{KL}(Q||P)$)

   Solution:

   (a) False. Overfitting isn't related to the optimizer that is used. Training rate is determined by the size / complexity of our model and the quantity of data available.

   (b) True / False. When the learning rate is suitably controlled, gradient descent would converge to a point with zero gradient. However, this may be a saddle point or a local minima.

   (c) False. A linear model as in the case of a single perceptron would not be enough to learn an XOR gate

   (d) False. Increasing model complexity may lead to overfitting on the training data and this lead to worse performance on the test set.

   (e) False. $\mathbb{KL}(P||Q) = \int p \log \frac{p}{q} \neq \int q \log \frac{q}{p} = \mathbb{KL}(Q||P)$

6. **[15 pts]** Consider the following neural network where we consider some activation function $\sigma(.)$ (**NOTE**: we do not apply activation on the output layer)

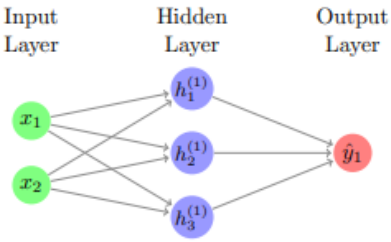


Figure 1: Schematic showing the neural network described in the question.

Answer the following questions with reference to the above network:

(a) **[10 pts]** Mathematically describe the forward pass of the network in terms of weight matrices (assume zero biases). If we were to train this model using gradient descent with the mean squared error, describe the update rules for the weight matrices of the neural network derived by back-propagation.

(b) **[5 pts]** Explain the significance of the Glorot initialization scheme.

Solution:

(a) We can mathematically describe the network as:

$$h^{(1)} = \sigma(W^{(1)}X) \tag{12}$$

$$\hat{y} = W^{(2)}h^{(1)} \tag{13}$$

We have that the loss term is given by:

$$J = \frac{1}{2}(y - \hat{y})^2 \tag{14}$$

Taking derivatives w.r.t the parameters of the network, we have:

$$\frac{\partial J}{\partial W^{(2)}} = -(y - \hat{y})\frac{\partial \hat{y}}{\partial W^{(2)}} = -(y - \hat{y})\frac{\partial (W^{(2)}h^{(1)})}{\partial W^{(2)}} = -(h^{(1)})^\top (y - \hat{y}) \tag{15}$$

$$\frac{\partial J}{\partial W^{(1)}} = -(y - \hat{y})\frac{\partial \hat{y}}{\partial W^{(1)}} = -(y - \hat{y})\frac{\partial \hat{y}}{\partial h^{(1)}}\frac{\partial h^{(1)}}{\partial W^{(1)}} = -X^\top (y - \hat{y})W^{(2)}\sigma'(W^{(1)}X) \tag{16}$$

$$\tag{17}$$

Update rule

$$W_1^{n+1} = W_1^n - \alpha\frac{dJ}{dW_1} \tag{18}$$

$$W_2^{n+1} = W_2^n - \alpha\frac{dJ}{dW_2} \tag{19}$$

(b) The idea behind the Glorot initialization is to set the initial weights of the network in such a way that the activations of each layer have roughly the same variance during forward propagation and backward propagation. This helps to avoid the problem of vanishing or exploding gradients, which can make it difficult to train deep neural networks.

4 Question 4 (15 pts)

Consider the following convolutional layer in a conv net. The input has a spatial dimension 12×12, and has 10 channels. The convolution kernels are 3×3, the stride is 2, and we use "valid" convolution, which means that each output neuron only looks at image regions that lie entirely within the spatial bounds of the input. The output dimension is 5×5, with 20 channels.
For this question, you don't need to show your work or justify your answer, but doing so

may help you get partial credit.
(a) How many weights are required for this convolution layer? (10 pts)
(b) Suppose we instead make this a locally connected layer, i.e. don't use weight sharing. How many weights are required? (5 pts)

(a) A convolutional layer with N input channels, M output channels, and $K \times K$ spatial extent requires MNK^2 weights. Hence, we need: $10 \times 20 \times 3 \times 3 = 1800$ weights.

(b) The locally connected layer has the same pattern of connections as the convolution layer but each of the $5 \times 5 = 25$ output locations will have its own separate set of weights. Hence, the total number of weights is $25 \times 1800 = 45000$.

Question 5 (10pts)

(10 pts) Please list out some different techniques you know for regularizing the training of deep neural networks (at least 4 and with some comments).

L1, L2, data augmentation, weight decay, dropout, early stopping, stochastic gradient descent (random batches)

5. **[10 pts]** Consider a simple CNN for an image classification problem:

- Input layer: Accepts $16 \times 16 \times 3$ images.
- Convolution layer: 6 filters, filter size $5 \times 5 \times 3$, stride 1, no padding.
- ReLU layer: Accepts and returns a $y \times y \times 6$ tensor.
- Max pooling layer: Pooling size 2×2, stride 2.
- Fully connected layer: Accepts a flattened tensor of dimension z and outputs a 10-dimensional tensor which encodes the predicted class labels.

Answer the following questions with reference to the above network:

(a) Determine y, z.

(b) Calculate the total number of trainable parameters involved in the neural network.

Solution:

(a) $y = 12, z = 216$

(b) Listing the number of trainable parameters considering each layer

- Convolution layer: 6 filters, filter size $5 \times 5 \times 3$, we have $n_{\text{weights}} = d_{\text{in}} \times f \times f \times d_{\text{out}} = 3 \times 5 \times 5 \times 6 = 450$ and $n_{\text{biases}} = d_{\text{out}} = 6$.
- Fully connected layer: Accepts a flattened tensor of dimension z and outputs a 10-dimensional tensor we have $n_{\text{weights}} = w \times 10 = 216 \times 10 = 2160$ and $n_{\text{biases}} = d_{\text{out}} = 10$.

3. **[5 pts]** Mike and Michelle have implemented two neural networks for recognizing handwritten digits from 16×16 grayscale images. Each network has a single hidden layer, and makes predictions using a soft-max output layer with 10 units, one for each digit class.

   - Michelle's network is a convolutional net. The hidden layer consists of three 16×16 convolutional feature maps, each with filters of size 5×5, and uses the logistic nonlinearity. All of the hidden units are connected to all of the output units.
   - Mike's network is a fully connected network with no weight sharing. The hidden layer consists of 768 logistic units (the same number of units as in Michelle's convolutional layer).

   Briefly explain one advantage of Michelle's approach and one advantage of Mike's approach.

Michelle's advantage: Due to weight sharing, Michelle's CNN has far fewer parameters, so it is less prone to overfitting and is likely to generalize better.

Mike's advantage: Each of Mike's hidden units sees the entire image, so it can learn global, location-specific patterns, whereas Michelle's hidden units can only detect local 5×5 patterns. (This is why deeper CNNs stack multiple convolutional layers to increase the receptive field so both local and global features can be learned.)